

Document number: P3472R0
Date: 2024-10-15
Audience: LEWG
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

Make `fiber_context::can_resume()` `const`

We propose to change the signature of function `fiber_context::can_resume()` so that it is a `const` member function.

[P0876R17] proposes a great library, `fiber_context`, providing an API for stackful context switching, based on [Boost.Context](#). It has one unusual design point in its interface. Member function `fiber_context::can_resume()`, which tells if the given fiber can be resumed by the calling thread, which is used as a precondition for other member functions in `fiber_context` is *not* declared `const`, implying that it cannot be called concurrently, without external synchronization, from multiple threads.

The older revision of the paper, [P0876R6], gives the rationale for making this function non-`const`.

`can_resume()` is not marked `const` because in at least one implementation, it requires an internal context switch. However, the stack operations are effectively read-only. Nonetheless, if it is possible for more than one thread to call `can_resume()` concurrently on the same non-empty `std::fiber_context` instance, locking is the caller's responsibility.

Given this potential data race, it is conceptually difficult to accept that it is a predicate suitable for use as a function precondition. In the context of the proposed contract support framework ([P2900R9]), function `fiber_context::can_resume()` would not be usable in contract assertions, as they always treat objects used in the predicates as `const`.

We believe that this "not `const`" provision is not necessary. The functionality of `can_resume()` can be implemented under a hosted implementation by storing the result of `std::this_thread::get_id()` upon the first fiber resumption and comparing it against the result of `std::this_thread::get_id()` performed inside `can_resume()`. This technique is used in the [reference implementation](#). Under a freestanding implementation `can_resume()` can be implemented as `!empty()`.

References

- [P0876R6] — Oliver Kowalke, Nat Goodspeed, "*fiber_context* - fibers without scheduler", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0876r6.pdf>).
- [P0876R17] — Oliver Kowalke, Nat Goodspeed, "*fiber_context* - fibers without scheduler", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p0876r17.pdf>).
- [P2900R9] — Joshua Berne, Timur Doumler, Andrzej Krzemiński et al., "Contracts for C++", (<https://isocpp.org/files/papers/P2900R9.pdf>).